

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0515

Slot :B

COURSE OUTCOME COVERED

CO3: *Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)*

Assessment Tool Weightage: 40%

QUESTIONS:3

Total Marks 30

Duration :60 Minutes

Annexure A

Experiments

Code of Conduct:

I am A.SHIVA SHANKAR REDDY(192425174) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

A.SHIVA SHANKAR REDDY

Signature of the student:

1. Harris Corner Detection Analysis

Perform Harris corner detection on images containing varying textures and patterns. Explain the concept of corner detection and its importance in computer vision tasks. Use appropriate parameters and tools to execute the experiment, document the detected corner points clearly, and analyze the distribution, accuracy, and significance of the detected corners.

CODE

```
import cv2

import numpy as np

from matplotlib import pyplot as plt

img = cv2.imread("input.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

gray = np.float32(gray)

# Harris Corner Detection

corners = cv2.cornerHarris(gray, 2, 3, 0.04)

# Dilate corner points

corners = cv2.dilate(corners, None)

img[corners > 0.01 * corners.max()] = [0, 0, 255]

plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))

plt.title("Harris Corner Detection")

plt.axis("off")

plt.show()
```

INPUT IMAGE:



OUTPUT IMAGE:



2. Effect of Noise on Corner Detection

Design an experiment to study the effect of noise on Harris corner detection performance. Explain the purpose of introducing noise and its influence on image features. Add noise systematically, perform corner detection using suitable tools, document observations for original and noisy images, and analyze the impact of noise along with possible improvement techniques.

PYTHON PROGRAM:

```
import cv2

import numpy as np

from matplotlib import pyplot as plt

img = cv2.imread("input.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

noise = np.random.normal(0, 25, gray.shape).astype(np.uint8)

noisy = cv2.add(gray, noise)

gray1 = np.float32(gray)

corner1 = cv2.cornerHarris(gray1, 2, 3, 0.04)

gray2 = np.float32(noisy)

corner2 = cv2.cornerHarris(gray2, 2, 3, 0.04)

original = img.copy()

original[corner1 > 0.01 * corner1.max()] = [0,0,255]

noisy_img = cv2.cvtColor(noisy, cv2.COLOR_GRAY2BGR)

noisy_img[corner2 > 0.01 * corner2.max()] = [0,255,0]

plt.figure(figsize=(10,5))

plt.subplot(1,2,1)

plt.imshow(cv2.cvtColor(original, cv2.COLOR_BGR2RGB))

plt.title("Original Image")

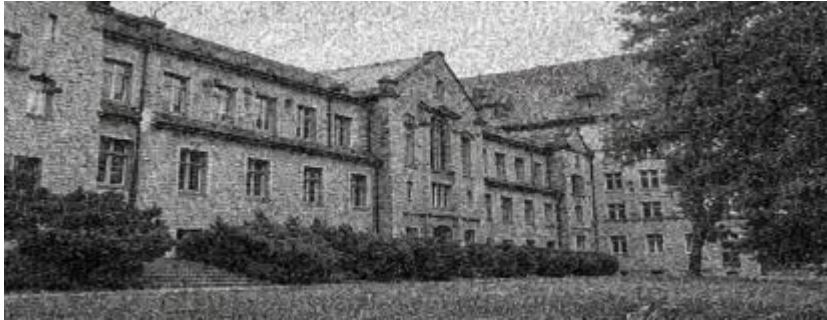
plt.subplot(1,2,2)

plt.imshow(cv2.cvtColor(noisy_img, cv2.COLOR_BGR2RGB))
```

```
plt.title("Noisy Image")
```

```
plt.show()
```

INPUT IMAGE:



OUTPUT IMAGE:



3. Hough Transform for Shape Detection

Conduct an experiment using the Hough Transform to detect lines or circles in an image. Explain the principles and significance of the Hough Transform in shape detection. Execute the detection process using appropriate tools, record the detected shapes and their parameters, and analyze the detection accuracy under different image conditions

PYTHON PROGRAM: Hough Transform (Line Detection)

```
import cv2

import numpy as np

img = cv2.imread("lines.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

edges = cv2.Canny(gray, 50, 150)

lines = cv2.HoughLines(edges, 1, np.pi/180, 150)

if lines is not None:

    for line in lines:

        rho, theta = line[0]

        a = np.cos(theta)

        b = np.sin(theta)

        x0 = a * rho

        y0 = b * rho

        x1 = int(x0 + 1000 * (-b))

        y1 = int(y0 + 1000 * (a))

        x2 = int(x0 - 1000 * (-b))

        y2 = int(y0 - 1000 * (a))

        cv2.line(img, (x1,y1), (x2,y2), (0,0,255), 2)

cv2.imshow("Detected Lines", img)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

INPUT IMAGE:



OUTPUT IMAGE:



Hough Circle Detection

Python PROGRAM

```
import cv2

import numpy as np

img = cv2.imread("circle.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```

gray = cv2.medianBlur(gray, 5)

circles = cv2.HoughCircles(

    gray,

    cv2.HOUGH_GRADIENT, 1,20,

    param1=100,

    param2=30,

    minRadius=10,

    maxRadius=100)

if circles is not None:

    circles = np.uint16(np.around(circles))

    for i in circles[0, :]:

        cv2.circle(img, (i[0], i[1]), i[2], (0,255,0), 2)

        cv2.circle(img, (i[0], i[1]), 2, (0,0,255), 3)

cv2.imshow("Detected Circles", img)

cv2.waitKey(0)

cv2.destroyAllWindows()

```

OUTPUT IMAGE:



Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	